

Hyunsik Jeon

Global AI Strategist

Portfolio: <https://portfolio-app-5ca9.vercel.app/>

jeonhs9110@gmail.com | Seoul, Korea

+82 10-4092-0628 | www.linkedin.com/in/jeonhyunsik | github.com/jeonhs9110



FEATURED PROJECT 01

AIIM — AI Influencer Match

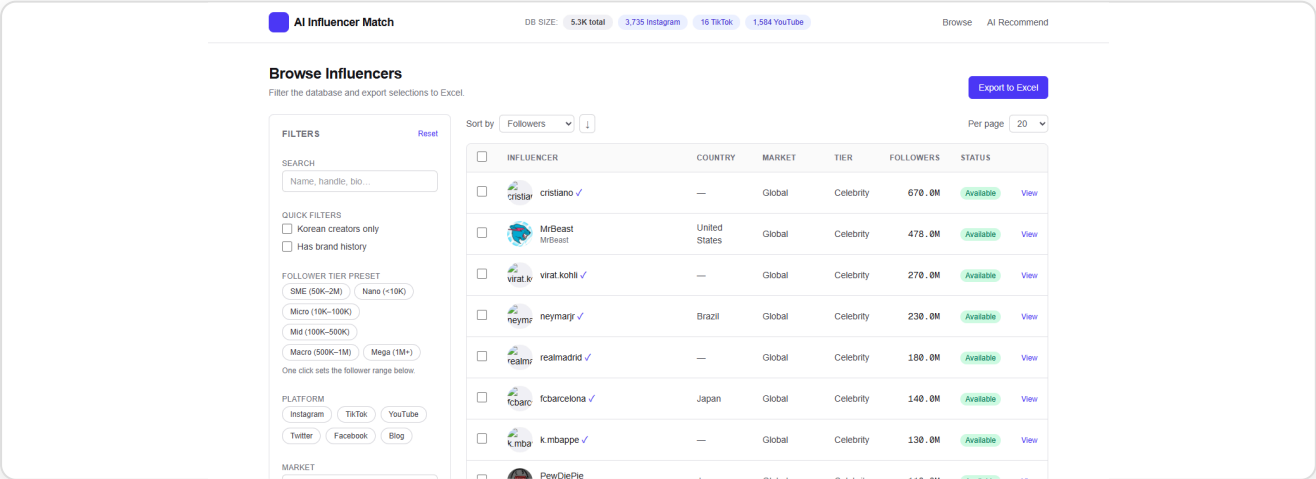
Hyunsik Jeon (Solo)

Python · FastAPI · Next.js 14 · LangGraph · GraphRAG · PostgreSQL (Cloud SQL) · Selenium · YouTube Data API · Meta Graph API · LLM · GCP Cloud Run

An end-to-end cloud platform that ingests 5,300+ creators across YouTube + Instagram + TikTok in 14 languages and **ranks brand-fit via a LangGraph agentic workflow layered on GraphRAG retrieval** — returning shortlists with per-pick rationale. Multilingual hashtag discovery surfaced 400 new YouTube creators in one 30-min run; extracted 323 IG business emails. Runs 24/7 on Google Cloud (Cloud Run + Cloud SQL). **Live:** aiim-frontend-559416574965.asia-northeast3.run.app · **GitHub:** github.com/jeonhs9110/AIIM-Showcase

Technical Pipeline Detail

- LangGraph agentic workflow** Brief parsing → candidate retrieval → multi-criteria scoring → rationale writing, modelled as a state graph (with retries, branching, human-in-the-loop hooks)
- GraphRAG retrieval** Creator · category · brand-collab relations encoded as a knowledge graph so every pick comes with a structural "why this creator" rationale
- Multi-platform ingestion** YouTube (1,584) · Instagram (3,735) · TikTok (16) — 5,300+ creators total
- Multilingual hashtag strategy** Searches 14 writing systems (광고 · スポンサー · رعاية ...) to escape English-query convergence
- LLM enrichment** Country · language · brand-collab history · vertical · contacts — auto-extracted with prompt caching
- Infra** GCP asia-northeast3 — Cloud Run + Cloud SQL Postgres + Cloud Build CI



FEATURED PROJECT 02

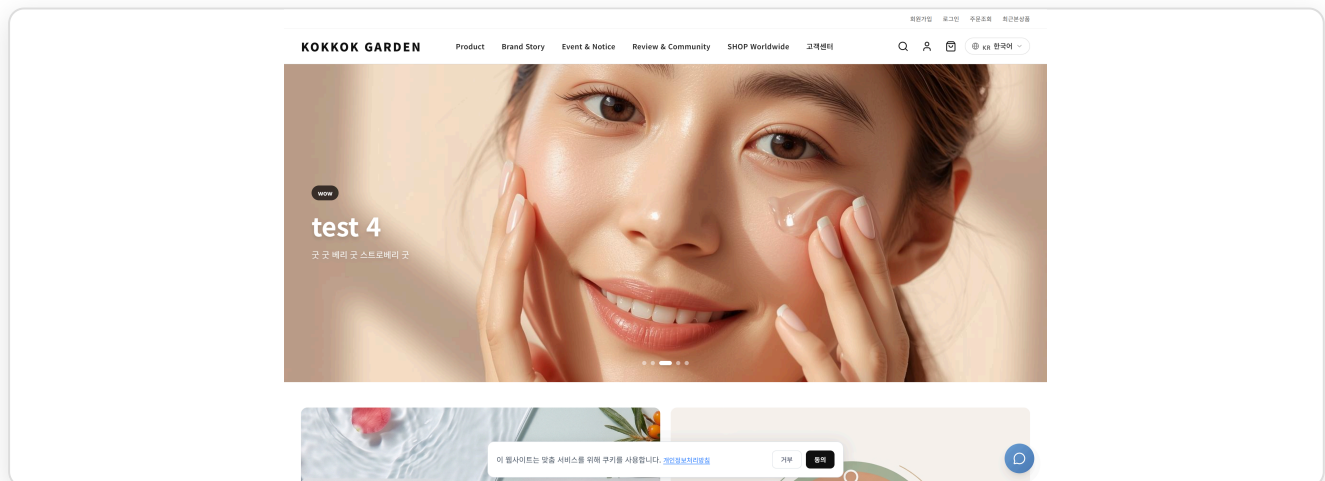
KOKKOK Garden — K-Beauty Multi-region E-commerce Hyunsik Jeon (Solo)

Next.js 16 · TypeScript · Supabase (PostgreSQL + Auth + Storage) · Tailwind CSS 4 · OpenAI GPT-4o mini · Vercel

A sustainable e-commerce designed for an administrator to keep running long after the initial build. Six core data entities (products · orders · cart · users · media_stories · YouTube Shorts) are normalised into a Supabase (PostgreSQL) schema under proper RLS and relational constraints; every entity has a unified `/admin` CRUD + image-upload + state flow so a non-developer can operate the store. KR (purchasable) / GL (viewing + AI chatbot) routing and GPT-4o mini 5-language product translation all run from one codebase. **Live:** <https://kokv2.vercel.app> · **GitHub:** github.com/jeonhs9110/kok_v2

Technical Pipeline Detail

- 1. Sustainable admin tooling** `/admin` dashboard — product · Shorts · media CRUD + image upload, operable without a developer
- 2. Normalised DB schema** Six entities on Supabase (PostgreSQL) — users · products · orders · cart_items · media_stories · shorts — with RLS and relational constraints
- 3. Unified auth** Supabase Auth + cookie-based admin session — login, permissions, and RLS handled in one layer
- 4. Multi-region routing** KR/GL split via `x-user-country`, six language routes
- 5. AI translation + chatbot** GPT-4o mini 5-language translation + unstable_cache 24h caching, with a K-beauty consultation chatbot on the GL store
- 6. Deploy** Vercel · <https://kokv2.vercel.app>



FEATURED PROJECT 03

IAMX6 — Naver Blog Automation Suite

Hyunsik Jeon (Solo · source private)

Electron · Puppeteer-core · Node.js · LLM · Bytenode (V8) · Electron Fuses · Windows portable .exe

Key impact: **a viral-marketing engine for Naver blog owners**. Automates neighbor requests · comment/like · visitor replies with human-shaped behavior to pile up the inbound / dwell / interaction signals the Naver algorithm rewards — deliberately pushing posts toward **top search ranking (검색 상위 노출)**. Distributed as a single ~67MB portable .exe, three-layer hardened (control-flow-flattened JS + V8 bytecode + Electron Fuses). **GitHub:** github.com/jeonhs9110/IAMX6-showcase

Technical Pipeline Detail

- 1. Viral-marketing impact** Accumulates inbound / dwell / interaction signals → maximises probability of **top search ranking (검색 상위 노출)** on Naver search and blog rankings
- 2. 3-workflow unified GUI** Neighbor-request · comment/like · per-visit reply + settings tab
- 3. Puppeteer automation** Drives a real Naver session — 2FA/CAPTCHA handled manually by the user, then the bot takes over
- 4. LLM comment generation** Post title + body → context-shaped Korean comments, 6–10 cached per run
- 5. Human simulation** Dwell time scaled to character count + $\pm 25\%$ jitter + ms-level noise + 35% "distraction" actions
- 6. 3-layer hardening** javascript-obfuscator → bytenode (.jsc) → Electron Fuses — blocks `asar extract`, debugger attach, and run-as-Node

FEATURED PROJECT 04

Top Award — IBM x RedHat AI Transformation Program

Vaccine Daily — AI News Intelligence Pipeline

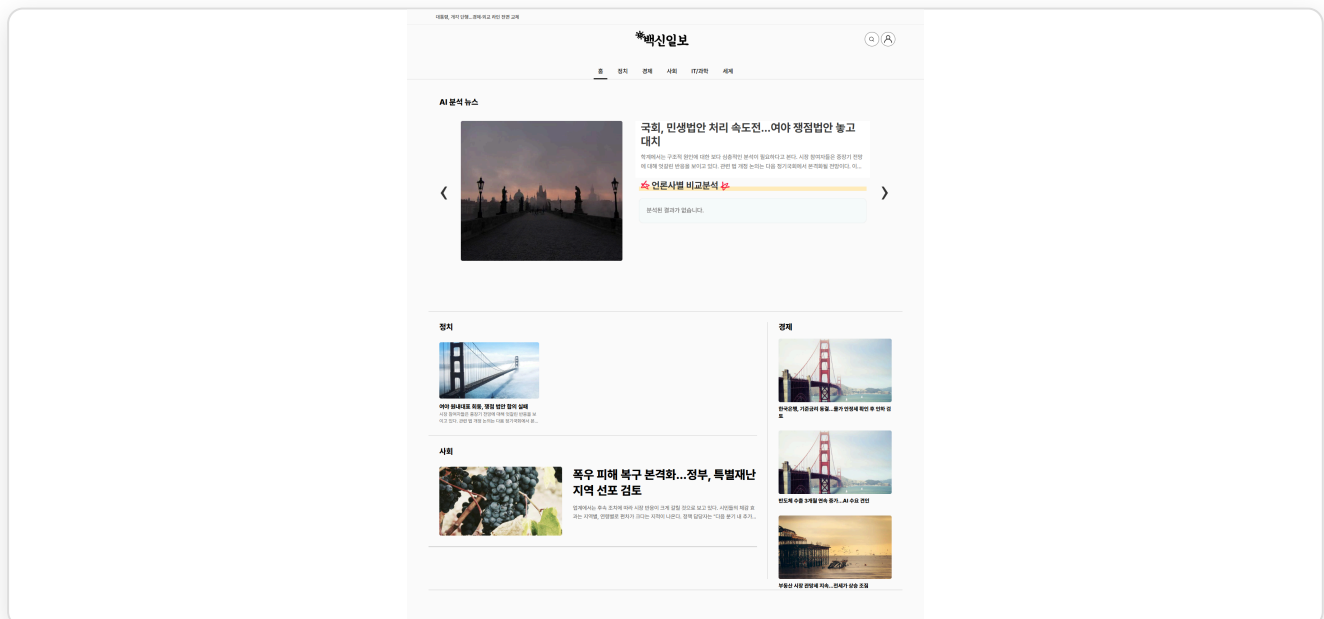
Hyunsik Jeon and 2 others

React · FastAPI · PostgreSQL · ChromaDB · HDBSCAN · ko-sroberta · Kiwi · KR-WordRank · GPT-4o-mini · GraphRAG · Docker · AWS EC2

Cutting-edge AI for cross-outlet reporting-tone comparison. A news-automation pipeline that systematically cuts LLM API cost by composing multiple Python libraries around GPT-4o-mini, and kills hallucinations with a Writer → Critic → Editor 3-agent system. HDBSCAN + ko-sroberta clustering eliminates duplicate LLM calls, a two-stage Kiwi morphological filter catches most obvious cases before the LLM, and a ChromaDB embedding cache prevents re-encoding. **Live:** jeonhs9110.github.io/vaccine-daily/ · **GitHub:** github.com/jeonhs9110/vaccine-daily

Technical Pipeline Detail

- 1. Multi-agent system** Writer → Critic → Editor — Critic judges the draft against the source as PASS/FAIL for hallucination/fact-distortion, auto-retry on failure, async parallel up to 5 at a time
- 2. LLM cost reduction via multiple Python libraries** HDBSCAN clustering + ko-sroberta embeddings + two-stage Kiwi morphological filter + ChromaDB embedding cache — blocks duplicate LLM calls and re-encoding
- 3. Data collection** Selenium + BeautifulSoup crawl 10 media outlets on a 5-minute cycle
- 4. GraphRAG tone analysis** Entity-Relation triples → knowledge graph visualising per-outlet reporting tone and framing; only the graph summary is injected into the prompt
- 5. Personalised recommendation** Subscribed keywords + read history + category preference → multi-criteria scoring + TextRank + KR-WordRank ensemble + D3 word cloud
- 6. Deploy** Docker Compose + Nginx + Certbot (HTTPS) + AWS EC2



FEATURED PROJECT 05

FOBO AI — European Football Prediction

Hyunsik Jeon (Solo)

Python · PyTorch · Transformer · Graph Attention Network · PPO RL · Dixon-Coles · XGBoost · LightGBM · Flask · Google Cloud

A Machine Learning and Reinforcement Learning pipeline for football-match prediction. End-to-end ML system chaining PyTorch Transformer + GAT + Dixon-Coles correction + PPO RL + XGBoost/LightGBM into an 8-stage pipeline. Across 13 European leagues: ~90% hit rate at ensemble $\geq 77\%$ + favourite filter; ~95% when the PPO agent independently agrees at $\geq 90\%$. Deployed on Google Cloud via a 2-VM architecture for ~\$26/month. **Live:** <http://34.64.147.124/> · **GitHub:** github.com/jeonhs9110/epl-showcase

8-Step System Architecture

- 1. Scrape** Parallel Flashscore crawl across 13 leagues (incremental re-runs)
- 2. Validation** Integrity checks catch broken scrapes before they poison training
- 3. Seq optimisation** Per-league look-back window (Premier ~10, Championship ~5)
- 4. DL training** Transformer + GAT + Dixon-Coles NLL — infers defensive posture in weak-vs-strong matchups
- 5. PPO RL** Action $\in \{\text{Home, Draw, Away, Pass}\}$ with Kelly-Criterion-shaped reward
- 6. Hybrid ensemble** XGBoost + LightGBM on DL embeddings
- 7. Calibration** Platt scaling aligns reported probability with empirical hit rate
- 8. Flask API** Models preloaded, <200 ms per request, GPT-4o-mini chatbot scoped to today's matches

Training Day

OVERVIEW

Home

PREDICTIONS

Predict Match

Daily Schedule

ANALYSIS

GNN Rankings

Optimal Threshold

RL Confidence

STRATEGY

Strategy

Strategy Lab

Bet History

SYSTEM

Project History

Run Update

Engine Online

Training Day v7

FOBO AI

Pipeline architecture + design rationale

SYSTEM ACTIVE

English한국어

FOBO AI

Football Outcome Prediction Optimiser

A multi-stage machine-learning pipeline that scrapes 13 European leagues, fuses transformer + graph-attention + gradient boosting models, and uses reinforcement learning to decide when a prediction carries enough statistical edge to act on.

Transformer + GATDixon-Coles NLLPPO Reinforcement LearningXGBoost + LightGBM

SYSTEM OVERVIEW

FOBO AI ingests historical results from 13 European leagues, trains a hybrid deep-learning + tree-based ensemble, and applies a reinforcement-learning agent to decide when a prediction carries enough statistical edge to act on. The final output is served via this web dashboard.

PROCESS	STAGE	MODULE	PURPOSE	RESULT / INSIGHT
①	Scraping	scrape_flashscore.py	Parallel scraping of results, odds, xG	13 CSV files (one per league) with match date, teams, final score, pre-match odds 1/X/2, and expected goals. Subsequent runs only fetch matches missing from the existing CSV — no re-scraping.
②	Validation	check_data.py	CSV integrity, column checks	Catches broken scrapes early (missing columns, bad dates, duplicate match IDs) before feeding training. No silent data rot.

Different leagues need different history depths. Premier League teams